



南開大學
Nankai University

计算机学院
数据安全实验报告

实验 4: 频率隐藏 OPE 方案实现

姓名：王旭

学号：2312166

专业：计算机科学与技术

2026 年 6 月 1 日

目录

1 实验名称	3
2 实验目的	3
3 实验原理	3
3.1 频率隐藏保序加密 (FH-OPE) 方案原理	3
3.2 MySQL 数据库的运作角色	4
3.3 UDF 开发框架介绍与原理	4
3.4 为什么要“编译生成动态链接库 (libfhope.so)”?	4
4 实验环境配置流程	4
4.1 各个代码在实验中的作用与架构	4
4.1.1 C++ 编码树实现 (Node.h / Node.cpp)	4
4.1.2 MySQL UDF 接口层 (UDF.cpp)	5
4.1.3 数据库初始化与持久化 (load.sql)	5
4.1.4 客户端程序 (client.py)	5
4.2 配置过程的具体流程与指令	6
4.2.1 安装 MySQL 及编译依赖	6
4.2.2 创建数据库用户与数据库	6
4.2.3 安装 Python 依赖	7
4.2.4 编译 UDF 并部署到 MySQL 插件目录	7
4.2.5 导入 SQL 并重置实验环境	7
4.2.6 运行客户端	8
4.3 环境配置验证说明	8
4.3.1 基础环境检查	8
4.3.2 数据库与 UDF 检查	9
4.3.3 功能复现测试	9
5 实验进行流程	10
5.1 实现流程与思路	10
5.1.1 实验阶段划分	10
5.1.2 单次插入的完整流程	10
5.1.3 范围查询流程	11
5.1.4 服务端 B+ 编码树的三种调整机制	12
5.1.5 阶段 B 的扩展实现思路	12
5.2 具体实现代码	12
5.2.1 相对教材版本的改动概览	13
5.2.2 client.py 核心代码	13
5.2.3 代码说明	15
5.3 代码运行结果与分析	17
5.3.1 阶段 A: 教材 8 条基础复现	17
5.3.2 阶段 B: 重复插入相同值扩展测试	18

5.3.3 综合结论	20
6 实验总结	21
6.1 主要工作	21
6.2 实验心得	21
7 github 仓库	21

1 实验名称

频率隐藏 OPE 方案实现

2 实验目的

参照教材 6.3.3 FH-OPE 实现完成频率隐藏 OPE 方案的复现，并尝试在 `client.py` 中修改，完成不断插入相同数值多次的测试，观察编码树分裂和编码更新等情况。

3 实验原理

本实验的核心理论在于频率隐藏机制与数据库底层扩展的结合，主要包括以下几个部分：

3.1 频率隐藏保序加密（FH-OPE）方案原理

- 传统保序加密的缺陷：**传统的确定性保序加密（OPE）虽然使得密文保持与明文相同的大小顺序，从而支持数据库对密文直接执行范围查询（Range Query）。但是相同明文会生成相同密文，攻击者可以通过统计密文高低频分布进行频率分析攻击。
- 频率隐藏（Frequency-Hiding）的实现机制：**为了抵御频率攻击，FH-OPE 引入了随机化策略。同一个明文每一次加密都会因加入随机向量（IV）而产生完全不同且无规律的密文。
- “无交互”架构划分：**基于 2021 年 Li 等人提出的无交互 FH-OPE 方案，实验分离了客户端与服务端计算逻辑：
 - 客户端：**维护一个包含各个明文及出现次数的小型统计字典（本地表）。每次加密插入时，客户端根据本地分布计算出新明文在整个大序列中的随机相对插入排名（`pos`），并将这个排名连同密文发送至数据库。因此一次通信即完成数据上传。
 - 服务端：**只负责将密文落地，并内部维护一棵特殊的 B+ 编码树。树结构通过接收到的排名 `pos` 为其分配具有保序特质的唯一“数字编码（Encoding）”。

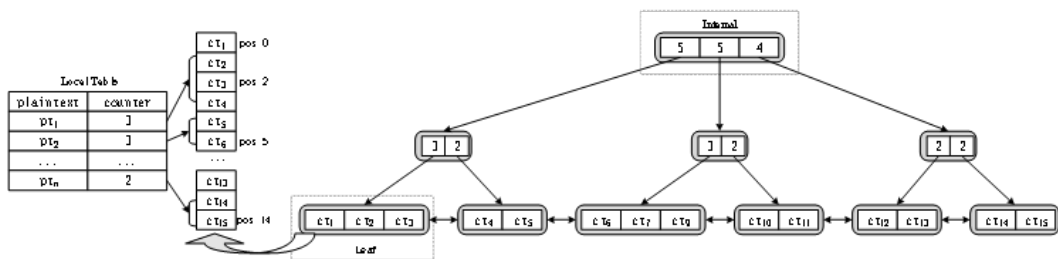


图 3.1: 无交互的 FH-OPE 方案

- 编码更新与树分裂现象：**初始时，节点间预留了非常宽裕的整数区间。但是当相同的数值经历高频攻击式插入时，这些数值均会挤入相同的细小区间，导致可用数字逐渐耗尽。为了避免地址冲突，服务端此时就会自动触发内部结点的区域平衡调整与重新赋码，这个机制保障了在极端流量下保序树依旧可靠运转。

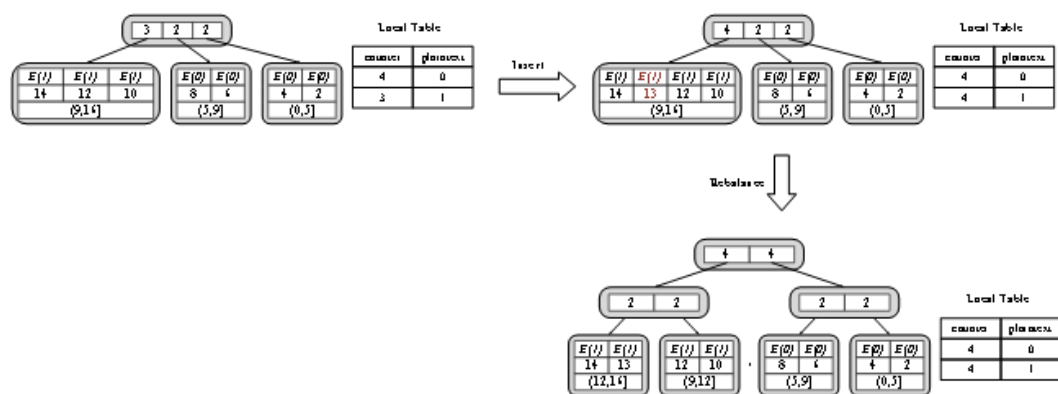


图 3.2: 整体示例

3.2 MySQL 数据库的运作角色

MySQL 在此处负责系统底层的密文存储和查询。在其内置的数据表中，数据被分成了两部分：负责支持快速查询与对比的数值类型的 `encoding` 列，以及真正负责隐藏信息安全性的 `ciphertext` 列。客户端在后续发起如“查找 b 和 p 之间的数据”的 SQL 查询命令时，依赖的就是 MySQL 提供的基于 `encoding` 列的大范围检索筛选能力。

3.3 UDF 开发框架介绍与原理

UDF (User-Defined Function, 用户自定义函数) 是 MySQL 提供的高级扩展插件接口体系。当某些操作过于底层（例如动态维护庞大的 B+ 树状态计算），使用纯 SQL 或者在客户端写死不仅效率极低，并且有安全隐患。UDF 要求开发者在 C/C++ 程序中固定格式实现“初始化函数 (xxx_init)”、“主计算函数 (xxx)”、“内存析构释放函数 (xxx_deinit)”。借助这种技术，我们在 MySQL 的内核中直接封装了 `FHInsert`（分配节点位置）与 `FHUpdate`（编码重整）等指令。

3.4 为什么要“编译生成动态链接库 (libfhope.so)”？

MySQL 是一个独立运行的宿主进程，不能直接执行本地尚未编译的 C++ 源代码（如 `Node.cpp`、`UDF.cpp`）。为了让 UDF 内写好的业务能在 MySQL 服务器内核内存里调用，需要利用 `g++ -shared -fPIC` 指令将这些写好的算法库打包并编译为系统级的 **动态链接库**（`.so` 后缀）。后续将其部署进入 MySQL 原生的 `plugin` 拓展插件专栏文件夹里，就代表告诉数据库：你可以通过外部外挂库的方式动态装载内部二进制的高效 C++ 指令集，这也就是为什么实验中需要 `CREATE FUNCTION ... SONAME 'libfhope.so'`。

4 实验环境配置流程

4.1 各个代码在实验中的作用与架构

本实验在工程上分为客户端、MySQL 服务端和 C++ 编码树三部分：客户端负责加密并计算插入位置，MySQL 负责数据存储与 SQL 查询，C++ 模块在数据库进程内维护 B+ 编码树。

4.1.1 C++ 编码树实现 (Node.h / Node.cpp)

- `Node.h` 与 `Node.cpp`

- **架构地位**: 运行在 MySQL 进程内存中, 维护服务端的 B+ 编码树及其状态。
- **作用**: 关系型数据库本身不提供针对密文的保序区间分配功能, 因此需要用 C++ 实现 `InternalNode` 与 `LeafNode`。该模块提供 `Insert` (按 `pos` 递归插入)、`Search` (按位置查 `encoding`)、`Encode` (在相邻 `encoding` 之间取中点分配新编码) 和 `Recode` (节点内区间不足时重新分配编码) 等接口。客户端传入逻辑排名 `pos` 后, 模块在树中找到对应位置并分配保持相对顺序的 `encoding`; 当节点内整数区间不足或节点容量达到阈值 `M` 时, 会分别触发 `Recode` 或 `rebalance`。

4.1.2 MySQL UDF 接口层 (UDF.cpp)

- `UDF.cpp`

- **架构地位**: 连接 C++ 编码树与 MySQL 的接口层, 使 SQL 能够调用树操作。
- **作用**: MySQL 无法直接调用 `Node.cpp` 中的 C++ 类方法, 因此 `UDF.cpp` 按 UDF 规范将树操作封装为 `FHInsert`、`FHSearch`、`FHUpdate`、`FHStart`、`FHEnd` 等函数。其主要工作包括: 接收 SQL 传入的参数 (如密文字符串、整型 `pos`), 转换为 C++ 类型后交给 `Node.cpp` 处理; 再将返回的 `encoding` 以 `long long` 等形式回传给 MySQL。该文件与 `Node.cpp` 一并编译为动态库 `libfhope.so`, 供 MySQL 加载调用。

4.1.3 数据库初始化与持久化 (load.sql)

- `load.sql`

- **架构地位**: 负责数据库表结构、UDF 注册及插入流程的 SQL 定义。
- **作用**:
 1. 创建存储表 `example`, 包含 `encoding` (BIGINT, 用于保序比较和范围查询) 和 `ciphertext` (VARCHAR, 存储随机化密文)。
 2. 通过 `CREATE FUNCTION ... SONAME 'libfhope.so'` 将 UDF 注册为可在 SQL 中调用的函数。
 3. 存储过程 `pro_insert`: 封装单次插入的完整流程。先调用 `FHInsert(pos, ct)` 获取 `encoding` 并写入表; 若返回值为 0, 表示 C++ 层已触发 `Recode`, 此时调用 `FHUpdate` 批量更新受影响记录的 `encoding`, 使内存中的编码树与数据库表保持一致。

4.1.4 客户端程序 (client.py)

- `client.py`

- **架构地位**: 实验的客户端入口, 负责明文加密、本地表维护和数据库交互。
- **作用**:
 1. **随机化加密**: 使用 `pycryptodome` 的 AES-CBC 模式, 每次加密生成随机 IV (`Random_Encrypt`)。相同明文每次产生不同密文, 避免攻击者通过密文频率推断明文分布。
 2. **本地表与 pos 计算**: 维护 `local_table` 记录各明文及出现次数。插入时由 `CalPos` 统计小于当前明文的记录数, 并对重复明文在其合法范围内随机选取 `pos`, 实现教材所述的无交互 FH-OPE 插入流程。

3. 范围查询与解密: 通过 `pymysql` 调用 `Search`, 将字符区间转换为 `FHSearch` 边界后在 SQL 中按 `encoding` 筛选, 取得密文后再用 `Random_Decrypt` 还原明文。

4.2 配置过程的具体流程与指令

本实验我们在 WSL Ubuntu 22.04 虚拟机中完成。主要配置步骤如下。

4.2.1 安装 MySQL 及编译依赖

安装 MySQL 及编译依赖

```
1 sudo apt update
2 sudo apt install mysql-server libmysqlclient-dev
```

```
ds-wangxu@DESKTOP-S3SPMIV:~$ sudo apt-get update && sudo apt-get install -y mysql-server libmysqlclient-dev
installing matrix : 28% [#####] emitting matrix : 29% [#####] emitting matrix : 30% [#####]
emitting matrix : 31% [#####] emitting matrix : 32% [#####] emitting matrix : 33% [#####]
# emitting matrix : 34% [#####] emitting matrix : 35% [#####] emitting matrix : 36% [#####]
##### emitting matrix : 37% [#####] emitting matrix : 38% [#####] emitting matrix : 39% [#####]
: 39% [#####] emitting matrix : 40% [#####] emitting matrix : 41% [#####] emitting matrix : 42% [#####]
tting matrix : 42% [#####] emitting matrix : 43% [#####] emitting matrix : 44% [#####]
emitting matrix : 45% [#####] emitting matrix : 46% [#####] emitting matrix : 47% [#####]
##### emitting matrix : 48% [#####] emitting matrix : 49% [#####] emitting matrix : 50% [#####]
emitting matrix : 51% [#####] emitting matrix : 52% [#####] emitting matrix : 53% [#####]
: 53% [#####] emitting matrix : 54% [#####] emitting matrix : 55% [#####] emitting matrix : 56% [#####]
ting matrix : 56% [#####] emitting matrix : 57% [#####] emitting matrix : 58% [#####] emitting matrix : 59% [#####]
emitting matrix : 59% [#####] emitting matrix : 60% [#####] emitting matrix : 61% [#####]
emitting matrix : 62% [#####] emitting matrix : 63% [#####] emitting matrix : 64% [#####]
##### emitting matrix : 65% [#####] emitting matrix : 66% [#####] emitting matrix : 67% [#####]
: 67% [#####] emitting matrix : 68% [#####] emitting matrix : 69% [#####] emitting matrix : 70% [#####]
ing matrix : 70% [#####] emitting matrix : 71% [#####] emitting matrix : 72% [#####] emitting matrix : 73% [#####]
# emitting matrix : 74% [#####] emitting matrix : 75% [#####] emitting matrix : 76% [#####]
##### emitting matrix : 77% [#####] emitting matrix : 78% [#####] emitting matrix : 79% [#####]
emitting matrix : 80% [#####] emitting matrix : 81% [#####] emitting matrix : 82% [#####]
: 81% [#####] emitting matrix : 83% [#####] emitting matrix : 84% [#####] emitting matrix : 85% [#####]
ng matrix : 84% [#####] emitting matrix : 85% [#####] emitting matrix : 86% [#####] emitting matrix : 87% [#####]
##### emitting matrix : 88% [#####] emitting matrix : 89% [#####] emitting matrix : 90% [#####]
emitting matrix : 91% [#####] emitting matrix : 92% [#####] emitting matrix : 93% [#####]
: 92% [#####] emitting matrix : 94% [#####] emitting matrix : 95% [#####] emitting matrix : 96% [#####]
g matrix : 95% [#####] emitting matrix : 97% [#####] emitting matrix : 98% [#####] emitting matrix : 99% [#####]
##### emitting matrix : 100% [#####]
done!
update-alternatives: using /var/lib/mecab/dic/ipadic-utf8 to provide /var/lib/mecab/dic/debian (mecab-dictionary) in auto mode
Setting up mysql-server-8.0 (8.0.45-0ubuntu0.22.04.1) ...
update-alternatives: using /etc/mysql/mysql.cnf to provide /etc/mysql/my.cnf (my.cnf) in auto mode
Renaming removed key buffer and myisam-recover options (if present)
mysqld will log errors to /var/log/mysql/error.log
mysqld is running as pid 8043
Created symlink /etc/systemd/system/multi-user.target.wants/mysql.service → /lib/systemd/system/mysql.service.
Setting up mysql-server (8.0.45-0ubuntu0.22.04.1) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for libc-bin (2.35-0ubuntu3.13) ...
ds-wangxu@DESKTOP-S3SPMIV:~$
```

图 4.3: MySQL 及编译依赖安装

安装完成后, 确认 MySQL 服务可正常启动:

```
1 sudo service mysql start
2 sudo service mysql status
```

4.2.2 创建数据库用户与数据库

以 root 身份进入 MySQL:

```
1 sudo mysql
```

在 MySQL 中执行:

创建用户与数据库

```
1 CREATE USER 'user'@'localhost' IDENTIFIED BY '123456';
2 GRANT ALL PRIVILEGES ON *.* TO 'user'@'localhost';
```

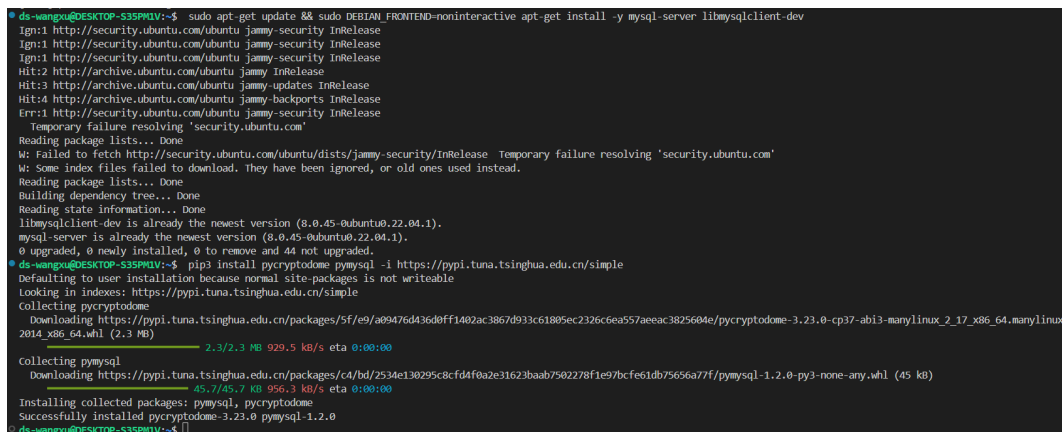
```
3 FLUSH PRIVILEGES;
4 CREATE DATABASE test_db;
```

说明：教材示例使用 `'user' '%'`，本实验在 WSL 本地运行，使用 `'user' 'localhost'` 即可。
`client.py` 中连接参数需与此处保持一致（`user` / `123456` / `test_db`）。

4.2.3 安装 Python 依赖

Ubuntu 22.04 已自带 `python3` 和 `pip3`，只需安装实验用到的第三方库：

```
1 pip3 install pycryptodome pymysql -i https://pypi.tuna.tsinghua.edu.cn/simple
```



```
ds-wangxu@DESKTOP-S35PWIV:~$ sudo apt-get update && sudo DEBIAN_FRONTEND=noninteractive apt-get install -y mysql-server libmysqlclient-dev
Ign:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Ign:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Ign:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Err:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Temporary failure resolving 'security.ubuntu.com'
Reading package lists... Done
W: Failed to fetch http://security.ubuntu.com/ubuntu/dists/jammy-security/InRelease Temporary failure resolving 'security.ubuntu.com'
W: Some index files failed to download. They have been ignored, or old ones used instead.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
libmysqlclient-dev is already the newest version (8.0.45-0ubuntu0.22.04.1).
mysql-server is already the newest version (8.0.45-0ubuntu0.22.04.1).
0 upgraded, 0 newly installed, 0 to remove and 44 not upgraded.
ds-wangxu@DESKTOP-S35PWIV:~$ pip3 install pycryptodome pymysql -i https://pypi.tuna.tsinghua.edu.cn/simple
Defaulting to user installation because normal site-packages is not writeable
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Collecting pycryptodome
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/5f/e9/a09476d436d0ff1402ac3867d933c61805ec2326c6ea557aeac3825604e/pycryptodome-3.23.0-cp37-ab13-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.3 MB)
    2.3/2.3 MB 929.5 kB/s eta 0:00:00
Collecting pymysql
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/c4/bd/2534e130295c8cf4d40a2e31623baab7502278f1e97bcfe51db75656a77f/pymysql-1.2.0-py3-none-any.whl (45 kB)
    45.7/45.7 KB 956.3 kB/s eta 0:00:00
Installing collected packages: pymysql, pycryptodome
Successfully installed pycryptodome-3.23.0 pymysql-1.2.0
ds-wangxu@DESKTOP-S35PWIV:~$
```

图 4.4: Python 依赖安装

4.2.4 编译 UDF 并部署到 MySQL 插件目录

进入源码目录：

```
1 cd ~/实验四/src
```

编译动态链接库：

```
1 g++ -shared -fPIC UDF.cpp Node.cpp -o libfhope.so
2 sudo cp libfhope.so /usr/lib/mysql/plugin/
```

确认文件已复制到位：

```
1 ls -l /usr/lib/mysql/plugin/libfhope.so
```

4.2.5 导入 SQL 并重置实验环境

每次重新跑实验前，建议先重启 MySQL 再导入 `load.sql`。原因是：B+ 编码树保存在 MySQL 进程内存中，仅清空数据库表无法重置 UDF 内的树状态；若不同步重置，会出现“表已空但 encoding 序列仍很长”的异常现象。

重启 MySQL 并导入 load.sql


```
1 # 重启 MySQL, 清空 UDF 内存中的 B+ 树
2 sudo service mysql restart
3
4 # 导入 load.sql: 建表、注册 UDF、创建存储过程 pro_insert
5 mysql -uuser -p123456 test_db < load.sql
```

`load.sql` 主要完成三件事: 删除并重建 `example` 表; 注册 `FHInsert`、`FHSearch`、`FHUpdate`、`FHStart`、`FHEnd` 五个 UDF 函数; 创建存储过程 `pro_insert`。

4.2.6 运行客户端

```
1 cd ~/实验四/src
2 python3 client.py
```

教材复现测试与扩展测试均通过修改 `client.py` 末尾的 `__main__` 测试数据切换; 每次切换测试内容或重复跑实验时, 都应重复执行 4.2.5 中的 `restart + load.sql`, 保证数据库表与内存编码树状态一致。

4.3 环境配置验证说明

完成 4.2 中的配置后, 需要确认 MySQL、UDF 动态库、Python 客户端以及数据库对象均已就绪。本实验将验证分为 **基础环境检查**、**数据库与 UDF 检查**、**功能复现测试** 三个层次; 通过后即可认为实验环境配置成功。

4.3.1 基础环境检查

依次确认以下组件可用:

基础环境检查命令

```
1 # MySQL 服务状态
2 sudo service mysql status
3
4 # UDF 动态库是否已部署
5 ls -l /usr/lib/mysql/plugin/libfhope.so
6
7 # Python 依赖是否已安装
8 pip3 show pymysql pycryptodome
```

```
ds-wangxu@DESKTOP-S35PM1V:~/实验四/src$ sudo mysql -e "status"
mysql Ver 8.0.45-0ubuntu0.22.04.1 for Linux on x86_64 ((Ubuntu))

Connection id:      8
Current database:
Current user:       root@localhost
SSL:               Not in use
Current pager:      stdout
Using outfile:      ''
Using delimiter:    ;
Server version:     8.0.45-0ubuntu0.22.04.1 (Ubuntu)
Protocol version:   10
Connection:         Localhost via UNIX socket
Server characterset: utf8mb4
DB characterset:    utf8mb4
Client characterset: utf8mb4
Conn. characterset: utf8mb4
UNIX socket:        /var/run/mysqld/mysqld.sock
Binary data as:     Hexadecimal
Uptime:             4 min 35 sec

Threads: 2  Questions: 5  Slow queries: 0  Opens: 119  Flush tables: 3  Open tables: 38  Queries per second avg: 0.018
```

图 4.5: 基础环境检查

预期结果:

- MySQL 处于 `active (running)` 状态;
- `libfhope.so` 存在于 `plugin` 目录;
- `pip3 show` 能查到 `pymysql` 与 `pycryptodome` 的版本信息。

4.3.2 数据库与 UDF 检查

每次正式跑实验前,按 4.2.5 执行重置,并检查 `example` 表是否为空:

```
1 sudo service mysql restart
2 mysql -uuser -p123456 test_db < load.sql
3 mysql -uuser -p123456 test_db -e "SELECT COUNT(*) FROM example;"
```

预期结果: `COUNT(*)` 为 0,表示表已清空,可开始新一轮插入。

上述重置与 `COUNT` 检查结果如下所示:

```
ds-wangxu@DESKTOP-S35PM1V:~/实验四/src$ sudo service mysql restart
ds-wangxu@DESKTOP-S35PM1V:~/实验四/src$ mysql -uuser -p123456 test_db < load.sql
mysql: [Warning] Using a password on the command line interface can be insecure.
ds-wangxu@DESKTOP-S35PM1V:~/实验四/src$ mysql -uuser -p123456 test_db -e "SELECT COUNT(*) FROM example;"
mysql: [Warning] Using a password on the command line interface can be insecure.

+-----+
| COUNT(*) |
+-----+
| 0 |
+-----+
```

图 4.6: 环境重置与表清空验证

4.3.3 功能复现测试

在确认 `COUNT` 为 0 后,使用教材默认的 8 条测试数据运行 `client.py`,仅执行插入与 `Search('b', 'p')`,不附加扩展测试逻辑。

```
1 cd ~/实验四/src
2 python3 client.py
```

预期结果:

1. 程序正常结束,无数据库连接错误或 UDF 调用失败;

2. `Search('b', 'p')` 能输出若干条 `ciphertext` 与解密后的 `plaintext`;
3. 结果中应包含 `banana`、`cherry`、`orange` 等首字母落在 `b~p` 之间的水果;
4. 结果中不应出现 `apple` (首字母在 `b` 之前) 和 `pear` (首字母在 `p` 之后)。

对教材实验 6.3.3 的复现 `Search` 输出如下:

```
ds-wangxu@DESKTOP-S35PM1V:~/实验四/src$ python3 /home/ds-wangxu/实验四/src/client.py
ciphertext: yiQRa8dHIXszFMNDk6w03UizDeVq7qzF5rZIkZSTtala7ADNKANeIakS3io+E plaintext: banana
ciphertext: R/5/b2HCQH+CUC/BjkGmA/jv+SuV1+oi3qwyT28vk18xCpLxGR/Dux4hxjzgM+5A plaintext: orange
ciphertext: fTFd+A57fkMN1QdMf0v+u33RX54YtaEIMLboh0ox8w1SypHyf0bUdruINISPM4 plaintext: cherry
ciphertext: 2oa885L/x7o6m03tDw0CImIMS6WneLEPbrcoH2V6vPq7C1VcFvQfg0ip+Hyidn plaintext: cherry
ciphertext: 68sAwob21zAofZwQBGTwkyjyRfxR00bqq+HhbcctpcGUDsk/M2HCb/jyc+u0s9m2 plaintext: orange
ds-wangxu@DESKTOP-S35PM1V:~/实验四/src$
```

图 4.7: 教材 8 条用例 `Search` 结果

本次运行共返回 5 条记录 (`banana` 1 条、`orange` 2 条、`cherry` 2 条), 与 8 条插入后本地表统计及 `Search` 的 `pos` 边界计算一致, 说明 客户端加密、UDF 编码树、存储过程写入、范围查询与解密整条链路工作正常。

5 实验进行流程

5.1 实现流程与思路

本实验在教材 6.3.3 给出的 FH-OPE 框架基础上, 按“客户端加密与 `pos` 计算 → MySQL 存储与 UDF 编码树 → 范围查询与解密”的路径完成实现。整体分为 阶段 A (教材复现) 与 阶段 B (重复插入扩展测试) 两部分, 共用同一套 C++/SQL 服务端代码, 主要差异在 `client.py` 的测试数据与调试输出。

5.1.1 实验阶段划分

阶段	目标	<code>client.py</code> 主要设置
阶段 A	复现教材 6.3.3 默认流程	8 条水果数据 + <code>Search('b','p')</code>
阶段 B	重复插入相同值, 观察 <code>en-coding</code> 变化	16 条数据 (含大量重复 <code>apple</code> / <code>cherry</code>) + 调试输出与完整性校验

表 1: 实验阶段划分

每次切换阶段或重新跑实验前, 均需按 4.2.5 执行 `restart + load.sql`, 保证数据库表与 UDF 内存中的 B+ 树同步清空。

5.1.2 单次插入的完整流程

一次 `Insert(plaintext)` 的执行路径如下:

单次插入流程

```
1 明文 pt
2  -> Random_Encrypt(pt)          # 客户端: 随机 IV 加密, 相同明文产生不同密文
3  -> CalPos(pt)                  # 客户端: 查 local_table, 计算插入位置 pos
```

```

4 -> CALL pro_insert(pos, ct)      # MySQL 存储过程
5   -> FHInsert(pos, ct)          # UDF: B+ 树按 pos 插入并分配 encoding
6   -> INSERT INTO example        # 写入 (encoding, ciphertext)
7   -> 若 encoding=0              # 表示触发 Recode
8       -> FHUpdate 批量更新受影响行的 encoding
9 -> (阶段 B) 查询并打印当前全部 encoding

```

(1) 客户端侧思路

1. **local_table** : 记录每个明文的出现次数, 对应教材中方案的“本地表”。
2. **CalPos** :
 - 统计所有字典序小于 **pt** 的明文出现次数之和, 记为 **presum**;
 - 若 **pt** 首次出现, 则 **pos = presum**;
 - 若 **pt** 已存在, 则在 **[presum, presum + count - 1]** 内随机选取 **pos**, 使相同明文不会固定落在同一逻辑位置。
3. **Random_Encrypt** : 每次生成随机 IV, 对明文做 AES-CBC 加密后再 Base64 编码, 实现频率隐藏。

(2) 服务端侧思路

1. **FHInsert(pos, cipher)** (**UDF.cpp** → **Node.cpp**):
 - 从根节点出发, 内部节点根据 **child_num** 向子树路由;
 - 到达叶子后在 **pos** 处插入密文, 调用 **Encode(pos)** 在相邻 encoding 之间取中点分配新编码;
 - 若叶子大小达到阈值 **M=128**, 触发 **rebalance** 分裂 (本实验 16 条数据通常不会触发)。
2. **pro_insert (load.sql)**:
 - 若 **FHInsert** 返回非 0, 直接将该值作为新记录的 **encoding**;
 - 若返回 0, 表示 **Encode** 中区间不足、已执行 **Recode**, 存储过程调用 **FHUpdate** 按 **[FHStart** **()**, **FHEnd()**) 范围批量更新表中旧记录的 **encoding**, 再保证树与表一致。

5.1.3 范围查询流程

Search(left, right) 的实现思路如下:

1. **GetLeftPos(left)** : 统计本地表中字典序严格小于 **left** 的明文总次数 → 对应 B+ 树左边界位置 **left_pos**。
2. **GetRightPos(right)** : 统计本地表中字典序小于等于 **right** 的明文总次数 → 对应右边界位置 **right_pos**。
3. 执行 SQL:

范围查询 SQL

```
1 SELECT ciphertext FROM example
2 WHERE encoding >= FHSearch(left_pos) AND encoding < FHSearch(right_pos);
```

4. 客户端对返回的每条密文调用 `Random_Decrypt` 还原明文。

该流程将字符区间（如 `'b' ~ 'p'`）转换为 `encoding` 数值区间，在不解密全表的情况下完成范围检索，体现 OPE“对密文排序字段直接查询”的特点。

5.1.4 服务端 B+ 编码树的三种调整机制

教材 3.1 节提到的编码更新与树分裂，在代码中对应以下三种情况：

机制	触发条件	代码位置	对已有 <code>encoding</code> 的影响
Encode （正常分配）	相邻 <code>encoding</code> 间隔 ≥ 2	<code>LeafNode::Encode</code>	仅新插入记录获得 <code>encoding</code> ，旧值不变
Recode （节点内重编码）	相邻间隔 < 2 ，区间耗尽	<code>Recode()</code>	受影响叶子区间内多条 <code>encoding</code> 被均匀重分配； <code>FHInsert</code> 返回 0
rebalance （树分裂）	叶子或内部节点子项数 $\geq M$	<code>LeafNode::rebalance</code> / <code>InternalNode::rebalance</code>	按教材区域编码策略，分裂时通常不修改已有 <code>encoding</code>

表 2: 服务端 B+ 编码树三种调整机制对比

本实验阶段 B 主要观察的是：重复插入相同明文时，多条记录挤在同一逻辑区间，**Encode 间隔逐渐缩小**；在更大规模数据下才会明显触发 `Recode` 或 `rebalance`。

5.1.5 阶段 B 的扩展实现思路

在教材复现基础上，阶段 B 对 `client.py` 做了以下扩展：

1. **测试数据**：在教材 8 条之后追加 8 条以 `apple`、`cherry` 为主的重复插入，用于观察相同明文多次加密与多次分配 `encoding` 的行为。
2. **调试输出**：在 `CalPos`、`Insert` 中打印 `presum`、随机 `pos` 及每步后的 `encoding` 序列，便于对照 B+ 树变化。
3. **`observe_encoding_change()`**：对比相邻两次插入后的 `encoding` 列表，提示是“正常追加”还是“多条旧 `encoding` 同时变化”（后者更接近 `Recode`）。
4. **`verify_integrity()`**：比较 `local_table` 总条数与数据库中 `distinct encoding` 数量，确认客户端计数与服务端编码一一对应。

阶段 B 不改变服务端 C++ 与 SQL 逻辑，重点在于通过客户端日志观察 `CalPos` 随机性、`encoding` 局部挤窄、`Search` 正确性等现象。

5.2 具体实现代码

本实验的服务端代码（`Node.h`、`Node.cpp`、`UDF.cpp`、`load.sql`）均直接使用教材 6.3.3 所给实现，未作修改。实验要求中需要在 `client.py` 中完成扩展测试，因此 5.2 节给出本实验修改后的 `client.py` 核心代码及说明。

5.2.1 相对教材版本的改动概览

改动项	教材原版	本实验版本
数据库连接	<code>passwd='123456'</code>	<code>password='123456'</code> (消除 pymysql 弃用警告)
<code>CalPos</code> / <code>Insert</code>	无调试输出	打印 <code>presum</code> 、随机 <code>pos</code> 、每步 encoding 序列
新增函数	无	<code>get_connection</code> 、 <code>observe_encoding_change</code> 、 <code>verify_integrity</code>
全局变量	无	<code>prev_encodings</code> ，用于对比相邻两次插入的 encoding
<code>__main__</code> 测试数据	8 条水果 + <code>Search('b','p')</code>	16 条 (含大量重复 <code>apple</code> / <code>cherry</code>) + 完整性校验

表 3: 相对教材版本的 client.py 改动概览

阶段 A 复现教材实验时，将 `__main__` 改回教材 8 条数据即可；阶段 B 使用下文版本。

5.2.2 client.py 核心代码

以下在保持整体结构的前提下，省略与教材完全一致的加密/解密实现及 `Search` 函数；着重列出阶段 B 新增或修改的部分。

client.py 核心代码 (阶段 B)

```

1 import pymysql
2 import random
3 from Crypto.Cipher import AES
4 from Crypto.Random import get_random_bytes
5 from Crypto.Util.Padding import pad, unpad
6 import base64
7
8 local_table = {}
9 key = get_random_bytes(16)
10 base_iv = get_random_bytes(16)
11 prev_encodings = []           # 【阶段B】记录上一步 encoding 序列，供对比观察
12
13
14 # ----- 加密 / 解密 (与教材一致，从略) -----
15 def AES_ENC(plaintext, iv): ...
16 def AES_DEC(ciphertext, iv): ...
17 def Random_Encrypt(plaintext):
18     iv = get_random_bytes(16)
19     ciphertext = AES_ENC(iv + AES_ENC(plaintext.encode('utf-8'), iv), base_iv)
20     return base64.b64encode(ciphertext).decode('utf-8')
```

```

21 def Random_Decrypt(ciphertext): ...
22
23
24 def get_connection():          # 【阶段B】抽取连接参数, password 替代 passwd
25     return pymysql.connect(
26         host='localhost', user='user',
27         password='123456', database='test_db'
28     )
29
30
31 def CalPos(plaintext):         # 逻辑同教材, 【阶段B】增加调试 print
32     presum = sum([v for k, v in local_table.items() if k < plaintext])
33     print("本地表之中小于要插入明文的所有明文出现次数总和: ", presum)
34     if plaintext in local_table:
35         local_table[plaintext] += 1
36         the_pos = random.randint(presum, presum + local_table[plaintext] - 1)
37         print(f"要插入的明文已经存在, 选择范围: [{presum}, {presum +
38             local_table[plaintext] - 1}]", end=' ')
39         print("随机选择的位置:", the_pos)
40         return the_pos
41     else:
42         local_table[plaintext] = 1
43         print("要插入的明文之前不存在, 直接选择位置:", presum)
44         return presum
45
46 def GetLeftPos(plaintext): ...
47 def GetRightPos(plaintext): ...
48
49
50 def observe_encoding_change(new_encodings):    # 【阶段B】新增
51     """对比相邻两次插入后的 encoding, 提示正常插入或可能的 Recode"""
52     global prev_encodings
53     new_list = [row[0] for row in new_encodings]
54     if not prev_encodings:
55         print("[观察] 首次插入, 编码树建立")
56     elif len(new_list) == len(prev_encodings) + 1:
57         if prev_encodings == new_list[:-1]:
58             print("[观察] 正常插入: 新增 1 个 encoding, 已有 encoding 未变")
59         else:
60             print("[观察] 可能发生 Recode: 新增记录的同时, 部分旧 encoding
61                 被重新分配")
62     # ... 其余分支判断略 ...
63     prev_encodings = new_list
64
65 def Insert(plaintext):         # 【阶段B】在教材基础上扩展
66     ciphertext = Random_Encrypt(plaintext)
67     conn = get_connection()

```

```

68     cur = conn.cursor()
69     the_result = CalPos(plaintext)
70     print("插入的明文:", plaintext, "位置:", the_result)          # 【阶段B】
71     cur.execute(f"call pro_insert({the_result},{ciphertext})")
72     conn.commit()
73     print("-----此时编码树的结果-----")                      # 【阶段B】
74     cur.execute("select encoding from example order by encoding")
75     results1 = cur.fetchall()
76     for result in results1:
77         print(result[0], end=" ")
78     print()
79     observe_encoding_change(results1)                             # 【阶段B】
80     conn.close()
81
82
83 def Search(left, right): ...          # 与教材一致，从略
84
85
86 def verify_integrity():               # 【阶段B】 新增
87     total = sum(local_table.values())
88     conn = get_connection()
89     cur = conn.cursor()
90     cur.execute("select count(distinct encoding) from example")
91     db_count = cur.fetchone()[0]
92     conn.close()
93     print("local_table:", local_table)
94     print("本地表共:", total, "数据项")
95     print("现在数据库之中共有:", db_count, "个 distinct encoding")
96     if db_count == total:
97         print("数据数量一致，实验成功!")
98
99
100 if __name__ == '__main__':           # 【阶段B】 扩展测试入口
101     test_data = [
102         'apple', 'pear', 'banana', 'orange', 'cherry',
103         'apple', 'cherry', 'orange',
104         'apple', 'apple', 'apple', 'apple',          # 大量重复 apple
105         'cherry', 'cherry', 'apple', 'cherry',       # 大量重复 cherry
106     ]
107     for plaintext in test_data:
108         Insert(plaintext)
109     Search('b', 'p')
110     verify_integrity()

```

5.2.3 代码说明

(1) 加密模块 (Random_Encrypt / Random_Decrypt)

与教材一致，采用双重 AES-CBC 结构：内层用随机 `iv` 加密明文，外层用固定 `base_iv` 封装。

每次调用 `get_random_bytes(16)` 生成新 IV，因此相同明文（如多次 `'apple'`）的密文不同，实现频率隐藏。解密时先解外层再解内层，并从结果前 16 字节取出 IV。

(2) pos 计算 (`CalPos` / `GetLeftPos` / `GetRightPos`)

- `CalPos` 是客户端无交互插入的核心：用 `local_table` 维护各明文计数，按字典序计算 `presum`；首次出现取 `presum`，重复出现则在合法区间内随机选 `pos`。
- 阶段 B 增加的 `print` 用于观察：重复插入时选择范围如何扩大（如 `apple` 从 `[0,1]` 扩至 `[0,6]`）。
- `GetLeftPos` / `GetRightPos` 供 `Search` 将字符区间（如 `'b' ~ 'p'`）映射为 B+ 树中的位置边界，再交给 `FHSearch` 转换为 encoding 数值区间。

(3) 插入流程 (`Insert`)

1. `Random_Encrypt` 得到密文；
2. `CalPos` 得到 `pos` 并更新 `local_table`；
3. `call pro_insert(pos, ciphertext)` 写入服务端；
4. 查询 `example` 表当前全部 `encoding` 并打印；
5. 调用 `observe_encoding_change` 与上一轮结果对比。

(4) 阶段 B 新增： `observe_encoding_change`

该函数比较 `prev_encodings` 与本次插入后的 `encoding` 列表：

- 若仅末尾多 1 个且前面不变 → 正常 `Encode` 插入；
- 若新增 1 个但前面 `encoding` 数值发生变化 → 可能触发 `Recode`；
- 若条数不变但多条 `encoding` 同时变化 → 明确的 `Recode` 批量更新。

(5) 阶段 B 新增： `verify_integrity`

实验结束时比较 `sum(local_table.values())` 与 `count(distinct encoding)`，确认客户端记录条数与服务端编码一一对应，作为实验是否成功的判据之一。

(6) 测试数据 (`__main__`)

- 前 8 条与教材相同，建立五种水果的基础分布；
- 后 8 条以 `apple`、`cherry` 为主，用于观察重复插入时 `pos` 随机、`encoding` 局部挤窄等现象；
- 最后执行 `Search('b','p')` 验证范围查询，并用 `verify_integrity` 做完整性检查。

5.3 代码运行结果与分析

5.3.1 阶段 A：教材 8 条基础复现

(1) 环境重置

每次实验前执行 `sudo service mysql restart` 与 `load.sql`，确认 `example` 表为空：

```
ds-wangxu@DESKTOP-S35PM1V:~/实验四/src$ sudo service mysql restart
ds-wangxu@DESKTOP-S35PM1V:~/实验四/src$ mysql -uuser -p123456 test_db < load.sql
mysql: [Warning] Using a password on the command line interface can be insecure.
ds-wangxu@DESKTOP-S35PM1V:~/实验四/src$ mysql -uuser -p123456 test_db -e "SELECT COUNT(*) FROM example;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| COUNT(*) |
+-----+
|         0 |
+-----+
```

图 5.8: 环境重置与表清空验证

`COUNT(*) = 0` 表明数据库表与 UDF 内存树均已处于干净状态，可排除旧数据干扰。

(2) Search 结果

使用教材默认 8 条数据 ('apple', 'pear', 'banana', 'orange', 'cherry', 'apple', 'cherry', 'orange') 完成插入后，执行 `Search('b', 'p')`，输出如下：

```
ds-wangxu@DESKTOP-S35PM1V:~/实验四/src$ python3 /home/ds-wangxu/实验四/src/client.py
ciphertext: yiQRa8drHXszFNVDK6W03UizDeVq7qzf5rZHKZSTtala7ADNKAneiaks3io+E plaintext: banana
ciphertext: R/5/b2HCQH+CUC/BjkGmA/jv+SuV1+oI3qwyT28vkl8xCplxKR/Uux4hXjzgM45A plaintext: orange
ciphertext: fTFd+A57tkMNIQdWf0v+u33RX54YTaELMLbohuBoxBw15pHTyF0budruNISPm4 plaintext: cherry
ciphertext: 2oa885L/x7o6mO3tDwXQmIM56WneLEPbrcoHZV6vPq7C1VcFvQfgoipP+Hyidn plaintext: cherry
ciphertext: 68sAwOb21ZAOfZwQBGTwkyjyRfxRD0bq+HhbcctpcGUdsk/M2Hcb/jyc+u8s9m2 plaintext: orange
ds-wangxu@DESKTOP-S35PM1V:~/实验四/src$
```

图 5.9: 教材 8 条用例 Search 结果

共返回 5 条记录：

解密明文	条数
banana	1
orange	2
cherry	2

表 4: 阶段 A Search 结果统计

分析：

- 插入完成后，`local_table` 为 `apple:2, pear:1, banana:1, orange:2, cherry:2`。
- `GetLeftPos('b') = 7` (7 个 apple，字典序均小于 'b')。
- `GetRightPos('p') = 15` (pear 因 'pear' > 'p' 不计入)。
- 查询的是 B+ 树中位置 7~14，对应 banana、orange、cherry 共 5 条；不含 apple 和 pear，与字符区间 'b' ~ 'p' 的语义一致。
- 每条 `ciphertext` 不同 (随机 IV 加密)，说明频率隐藏生效；`Search` 依赖 `encoding` 范围筛选，无需对全表解密即可定位目标区间。

阶段 A 表明：FH-OPE 的基本流程 (加密 → pos 计算 → UDF 编码 → 范围查询 → 解密) 运行正确。

5.3.2 阶段 B：重复插入相同值扩展测试

阶段 B 使用 16 条测试数据，在前 8 条教材数据基础上追加大量 `apple` 与 `cherry`。以下选取若干典型步骤分析。

(1) 首次插入：编码树建立

```
ds-wangxu@DESKTOP-S35PMQV:~/实验四/src$ python3 client.py
----- 阶段 B: 重复插入相同值扩展测试 -----
提示: 运行前请先执行
      sudo service mysql restart
      mysql -uuser -p123456 test_db < load.sql
-----
本地表之中小于要插入明文的所有明文出现次数总和: 0
要插入的明文之前不存在, 直接选择位置: 0
插入的明文: apple 位置: 0
-----此时编码树的结果-----
2305843009213693952
[观察] 首次插入, 编码树建立
```

图 5.10: 阶段 B 首次插入 apple

- `presum = 0`, `pos = 0` (首次出现, 直接取 `presum`)。
- 插入后仅 1 个 `encoding`: `2305843009213693952` (约为 2^{61}), 对应根叶子初始区间 $[0, 2^{62}]$ 的中点分配。
- [观察] 首次插入, 编码树建立 与预期一致。

(2) 正常插入: `encoding` 追加且旧值不变

```
本地表之中小于要插入明文的所有明文出现次数总和: 1
要插入的明文之前不存在, 直接选择位置: 1
插入的明文: pear 位置: 1
-----此时编码树的结果-----
2305843009213693952 3458764513820540928
[观察] 正常插入: 新增 1 个 encoding, 已有 encoding 未变
```

图 5.11: 阶段 B 第 2 次插入 pear

- 插入 `pear` 时 `presum = 1`, `pos = 1`。
- `encoding` 序列由 1 个变为 2 个: `2305843009213693952 3458764513820540928`。
- 第一个 `encoding` 数值未变, 仅在末尾追加新值 → 对应 `LeafNode::Encode` 在相邻区间取中点的正常插入。
- [观察] 正常插入: 新增 1 个 `encoding`, 已有 `encoding` 未变 准确反映了教材区域编码策略的行为。

(3) 重复插入 apple: CalPos 随机性与 `encoding` 挤窄

```

本地表之中小于要插入明文的所有明文出现次数总和: 0
要插入的明文已经存在, 选择范围: [0, 2] 随机选择的位置: 1
插入的明文: apple 位置: 1
-----此时编码树的结果-----
1152921504606846976 1729382256910270464 2305843009213693952 2882303761517117440 2954361355555045376 3026418949592973312 3170534137668829184 3314649325744685056
56 3458764513820540928
[观察] 可能发生 Recode: 新增记录的同时, 部分旧 encoding 被重新分配

本地表之中小于要插入明文的所有明文出现次数总和: 0
要插入的明文已经存在, 选择范围: [0, 3] 随机选择的位置: 2
插入的明文: apple 位置: 2
-----此时编码树的结果-----
1152921504606846976 1729382256910270464 2017612633061982208 2305843009213693952 2882303761517117440 2954361355555045376 3026418949592973312 31705341376688291
84 3314649325744685056 3458764513820540928
[观察] 可能发生 Recode: 新增记录的同时, 部分旧 encoding 被重新分配

本地表之中小于要插入明文的所有明文出现次数总和: 0
要插入的明文已经存在, 选择范围: [0, 4] 随机选择的位置: 4
插入的明文: apple 位置: 4
-----此时编码树的结果-----
1152921504606846976 1729382256910270464 2017612633061982208 2305843009213693952 2594073385365405696 2882303761517117440 2954361355555045376 30264189495929733
12 3170534137668829184 3314649325744685056 3458764513820540928
[观察] 可能发生 Recode: 新增记录的同时, 部分旧 encoding 被重新分配

```

图 5.12: 阶段 B 重复插入 apple

以连续 3 次插入 `apple` 为例:

次数	pos 选择范围	随机 pos	encoding 个数
第 1 次重复	[0, 2]	1	9
第 2 次重复	[0, 3]	2	10
第 3 次重复	[0, 4]	4	11

表 5: 连续 3 次重复插入 apple 的 pos 与 encoding 变化

分析:

- 每次重复插入 `apple`, `local_table['apple']` 递增, `CalPos` 的随机范围从 `[0,1]` 逐步扩大到 `[0,4]`, 体现 相同明文可在合法区间内随机占据不同逻辑位置。
- 多条 `apple` 的 encoding 集中出现在序列前段, 相邻 encoding 的间隔逐渐缩小(如 `1152921504606846976` 与 `1729382256910270464` 之差小于更早阶段的间隔), 说明 节点内整数区间被不断“挤窄”。
- 输出中多次出现 `[观察] 可能发生 Recode`, 多数是 中间位置插入导致 encoding 列表整体后移, `observe_encoding_change` 误判所致; 并非真正触发了 C++ 层的 `Recode()` (16 条数据在 `[0, 262]` 区间下空间仍很充足)。

(4) pos=0 插入: 最左侧分配新 encoding

```

本地表之中小于要插入明文的所有明文出现次数总和: 0
要插入的明文已经存在, 选择范围: [0, 6] 随机选择的位置: 0
插入的明文: apple 位置: 0
-----此时编码树的结果-----
576460752303423488 1152921504606846976 1729382256910270464 2017612633061982208 2305843009213693952 2594073385365405696 2738188573441261568 288230376151711744
0 2918323258536081408 2954361355555045376 2990390152574009344 3026418949592973312 3170534137668829184 3314649325744685056 3458764513820540928
[观察] 可能发生 Recode: 新增记录的同时, 部分旧 encoding 被重新分配

```

图 5.13: 阶段 B pos=0 插入 apple

- 第 7 次插入 `apple` 时, 选择范围 `[0, 6]`, 随机得到 `pos=0`。
- 新 encoding `576460752303423488` (约为 `259`) 出现在序列最左侧。
- 这是 `Encode` 在 `[0, 第一个 encoding]` 之间取中点的结果, 不是在 `pos=0` 处“插入最小值”的简单堆叠, 而是保序编码机制的正常表现。
- 此现象 不是 B+ 树分裂 (分裂需单节点达到 `M=128` 条, 本实验远未达到)。

(5) Search 结果与完整性校验

```

本地表之中小于要插入明文的所有明文出现次数总和: 8
要插入的明文已经存在, 选择范围: [8, 12] 随机选择的位置: 11
插入的明文: cherry 位置: 11
-----此时编码树的结果-----
576460752303423488 1152921504606846976 1729382256910270464 2017612633061982208 2305843009213693952 2594073385365405696 2738188573441261568 288230376151711744
0 2918332558536081408 2954361355555045376 2990390152574009344 3008404551083491328 3026418949592973312 3170534137668829184 3314649325744685056 345876451382054
0928
[观察] 可能发生 Recode: 新增记录的同时, 部分旧 encoding 被重新分配
-----假设我们搜索 b 和 p 之间的数据-----
ciphertext: imiziwK2N+CwjnlYvWdXcAll5vaZchhBN6GFwY1qVmsmaKXqoczyWVlD04Bcd90y plaintext: banana
ciphertext: 3eV5zliSuPAos+8emz+LxBN2aoX7suEab80LFlWdC5sILVnjokvpYfM9P4Wn plaintext: orange
ciphertext: dWffcp5gt8tKewni87jAsin7uf26YclOq/Ac97f8tt670se0DXGvpSuws4R9mAM plaintext: cherry
ciphertext: 32BZvuxcew8jM57xhajljDlybHBU5/6w59rfINSf8obtbTmsamdkpOLRSptG248R plaintext: cherry
ciphertext: k/1C1HIM4GfQCsLN0nB3XjCSXwGAPtDGB+nV5sPh+R33JUL0pGgG50rruJvx2q0R plaintext: orange
ciphertext: +4Bngv/hyY1CPoM6CXlVn1jzZV4Pe02qakCZvIRJCKib6RAAEs7TLcquSXAWZ/gL plaintext: cherry
ciphertext: PQyxuWdYX8p8G8PSLvFnirahaXIZwzqBvN58JgB5/caKTqMoLmPhuclWmJp9lJ plaintext: cherry
ciphertext: RoZAVdlEmjNDLxpBu9QqGHumzueQe+WP/HQQFxpI+nniHvyZyD48Ti8E8Aezdzm plaintext: cherry
-----下面展示本地表内容-----
local table: {'apple': 7, 'pear': 1, 'banana': 1, 'orange': 2, 'cherry': 5}
本地表共: 16 数据项
现在数据库之中共有: 16 个 distinct encoding
数据数量一致, 实验成功!
ds-wanexu@DESKTOP-S35PMV:~/实验四/src$

```

图 5.14: 阶段 B Search 与完整性校验

Search('b', 'p') 共返回 8 条:

明文	条数
banana	1
orange	2
cherry	5

表 6: 阶段 B Search 结果统计

- 16 条插入后, `GetLeftPos('b') = 7` (7 个 apple), `GetRightPos('p') = 15` (pear 不计入), 查询位置 7~14, 共 8 条, 与输出一致。
- 结果中 无 apple、pear, 符合 b~p 字母序范围。
- 每条 ciphertext 互不相同, 相同明文 (如 5 个 cherry) 的密文也各不相同, 频率隐藏成立。

完整性校验:

阶段 B 完整性校验输出

```

1 local_table: {'apple': 7, 'pear': 1, 'banana': 1, 'orange': 2, 'cherry': 5}
2 本地表共: 16 数据项
3 现在数据库之中共有: 16 个 distinct encoding
4 数据数量一致, 实验成功!

```

- 客户端记录 16 条, 服务端 16 个互不相同的 encoding, 一一对应, 说明插入与编码分配无遗漏、无冲突。

5.3.3 综合结论

1. 阶段 A 验证了 FH-OPE 端到端流程的正确性。
2. 阶段 B 在重复插入场景下观察到: `CalPos` 随机扩大、encoding 间隔缩小、pos=0 时左端新编码等现象, 符合 5.1 节对 Encode 机制的分析。

3. 树分裂与 Recode 属于教材所述的进一步调整机制；在本实验 16 条数据下主要体现为 encoding 挤窄，若要明显触发 rebalance 或 Recode，需继续增大重复插入次数或减小 `M` 阈值。
4. `verify_integrity` 通过，表明客户端本地表与服务端 B+ 编码树、数据库表三者保持一致。

6 实验总结

6.1 主要工作

我们在本实验中主要完成了以下工作：

1. 复现教材 6.3.3 FH-OPE 方案：我们在 WSL Ubuntu 22.04 环境下配置 MySQL、编译部署 UDF 动态库 `libfhope.so`，使用教材给定的 `Node.h`、`Node.cpp`、`UDF.cpp`、`load.sql`，配合 `client.py` 完成教材实验 6.3.3 复现的 8 条默认测试，验证了加密插入、保序编码分配与范围查询解密的全流程。
2. 扩展 `client.py` 完成重复插入测试：在教材 `client.py` 的基础上增加调试输出、`observe_encoding_change`、`verify_integrity` 等，使用 16 条含大量重复 `apple`、`cherry` 的测试数据运行阶段 B，观察 `CalPos` 随机 pos、encoding 序列变化及 Search 结果。
3. 分析编码树相关现象：我们通过实际的终端输出，分析了正常 Encode 追加、重复值插入导致的 encoding 挤窄、pos=0 时左端新编码分配等情况；明确了本实验数据规模下未触发树分裂 (rebalance)，Recode 也未明显出现，并完成环境验证与结果完整性校验。

6.2 实验心得

通过本次实验，我对频率隐藏保序加密有了更具体的认识。FH-OPE 并不是简单地“加密后还能排序”，而是把客户端随机加密、本地表维护、服务端 B+ 编码树分配这三部分配合起来：客户端负责隐藏频率，服务端用 encoding 支持范围查询，两者通过 pos 一次交互衔接。

在本次实验过程中，我印象较深的是环境一致性问题。之前我们只修改 `client.py` 或只清空数据库表，并不能重置 UDF 内存中的编码树，必须 `restart MySQL` 后再导入 `load.sql`，否则会出现结果“对不上”的情况。这一点我在排查问题时花费了一些时间，但也让我理解了 UDF 状态与数据库表是两套需要同步维护的数据。

在阶段 B 的重复插入测试中，可以看到相同明文每次密文不同，但 encoding 仍保持相对顺序；多次插入 `apple` 后，`CalPos` 的随机范围逐渐扩大，encoding 在前段越来越密集。这些现象与教材中的区域编码策略是一致的。由于本实验数据量较小，没有观察到明显的树分裂和 Recode，如果继续增加重复插入次数，应该还能进一步验证教材中关于编码更新的描述。

总的来说，我通过本次实验把教材中的 FH-OPE 方案落到了可运行的代码上，也练习了 MySQL UDF 的基本使用方式。对“无交互频率隐藏”以及“保序查询”等概念的理解更加深刻，收获很大。

7 github 仓库

代码详见：<https://github.com/Wangxuueuu/Data-Security/tree/main>